Kubernetes-Based Canary Deployment Using K3s and Istio

DevOps / Cloud Deployment Project

1. Introduction

In modern software development, applications need frequent updates without affecting existing users. Traditional deployment methods may cause downtime or failures when a new version is released.

Canary deployment is a progressive release strategy in which a new application version is introduced to a small percentage of users first. If the new version works correctly, traffic is gradually increased.

This project implements a Kubernetes-based Canary Deployment using K3s and Istio. It uses stable and canary application versions, weighted traffic routing, health checks, automated promotion, and rollback functionality.

The project was deployed and tested in a local K3s Kubernetes cluster.

2. Abstract

The main objective of this project is to implement a safe and controlled application deployment strategy using Kubernetes and Istio.

Two application versions are maintained:

- Stable version
- Canary version

Kubernetes manages the application deployments and services. Istio manages traffic distribution between the stable and canary versions using weighted routing.

An automated Bash script performs health checks and promotes the canary version through three stages:

- 80% Stable / 20% Canary

- 50% Stable / 50% Canary
- 0% Stable / 100% Canary

If a health check fails, the script is designed to roll back traffic to the stable version. The implementation was successfully tested on a local K3s cluster.

3. Tools and Technologies Used

Technology	**Purpose**
Docker	Containerization
Kubernetes	Container orchestration
K3s	Lightweight Kubernetes cluster
Istio	Service mesh and traffic management
Helm	Kubernetes package management
Bash	Automation and health checks
kubectl	Kubernetes resource management
Git	Version control
GitHub	Source code hosting and documentation
GitHub Actions	CI/CD workflow support

4. Steps Involved in Building the Project

4.1 Step 1: Environment Setup

Installed and configured Docker, K3s, Kubernetes CLI, Helm, and Istio.

4.2 Step 2: Containerization

Prepared the application container image using Docker.

4.3 Step 3: Kubernetes Deployments

Created separate stable and canary deployments inside the `canary-demo` namespace.

4.4 Step 4: Kubernetes Services

Configured Kubernetes Services to provide network access to the application pods.

4.5 Step 5: Istio Configuration

Configured Istio DestinationRule and VirtualService resources to manage stable and canary traffic routing.

4.6 Step 6: Weighted Traffic Routing

Implemented progressive traffic distribution:

- 80% Stable / 20% Canary
- 50% Stable / 50% Canary
- 0% Stable / 100% Canary

4.7 Step 7: Helm Integration

Used Helm to package and manage Kubernetes deployment configurations.

4.8 Step 8: Automated Canary Promotion

Created a Bash script named `auto-promote-canary.sh` to perform health checks and automatically update traffic weights.

4.9 Step 9: Rollback Implementation

Added rollback logic to restore 100% traffic to the stable version when health validation fails.

4.10 Step 10: Testing and Validation

Tested application responses through an in-cluster test client and verified the traffic routing behavior.

4.11 Step 11: GitHub Documentation

Uploaded project source code, scripts, configuration files, and documentation to GitHub.

5. Testing and Results

The project was tested successfully in the local K3s cluster.

Test Stage	Stable	Canary	Result
Initial stage	80%	20%	5/5 health checks passed
Intermediate stage	50%	50%	5/5 health checks passed
Final stage	0%	100%	5/5 health checks passed
Final request test	0%	100%	20/20 requests returned Canary

Note: These results are from local K3s testing. The application is not currently hosted publicly on AWS or another cloud provider.

The promotion script performs staged health checks. It is not a continuous monitoring controller.

6. Conclusion

This project demonstrates how Kubernetes and Istio can be used to implement a controlled Canary Deployment strategy.

The project successfully demonstrates:

- Containerization
- Kubernetes deployments
- Service management
- Istio traffic routing
- Helm integration
- Automated health checks
- Rollback functionality

The staged promotion process helps reduce deployment risks by gradually shifting traffic from the stable version to the canary version.

Through this project, practical knowledge was gained in DevOps tools, container orchestration, service mesh architecture, automation, and progressive application delivery.

7. Future Enhancements

- Integrate Prometheus and Grafana for monitoring.
- Implement metric-based automated promotion.

- Extend CI/CD automation.
- Deploy the application to a public cloud environment.
- Configure HTTPS and a public application endpoint.

8. GitHub Repository

GitHub:

https://github.com/sindhupriya5e1/kubernetes-canary-deployment-k3s-istio